

Relazione di Progetto: Sparse matrix-vector multiplication in High-Level Synthesis (HLS)

Stefano Scarcelli

Indice

1. Introduzione e Analisi degli Obiettivi	1
1.1. Descrizione del Progetto	1
1.2. Sfida principale (loop a iterazioni variabile)	1
1.3. Obiettivi	1
2. Metodologia di Esplorazione e Automazione del Workflow	2
2.1. Workflow automatizzato	2
2.2. Approccio all'analisi	2
3. Metriche di Valutazione e Trade-off	2
4. Esplorazione dello Spazio di Design	3
4.1. Analisi dell'Architettura Baseline	3
4.1.1. Implementazione Logica di Base	3
4.1.2. Testbench C/C++ Integrata	4
4.1.3. Disabilitazione delle Ottimizzazioni Automatiche	4
4.1.4. Analisi del Testbench VHDL e dei Constraint di Sistema	4
4.1.5. Analisi delle Metriche di Valutazione e Limitazioni	5
4.2. Architetture basi	7
4.2.1. Introduzione del Parallelismo Temporale (Loop Pipelining su L2)	7
4.2.2. Analisi delle Metriche di Valutazione e Limitazioni	7
4.2.3. Parallelismo Temporale sul Ciclo Esterno (Loop Pipelining su L1)	10
4.2.4. Analisi delle Metriche e Fallimento Strutturale	10
4.2.5. Parallelismo Spaziale sul Ciclo Esterno (Loop Unroll Parziale su L1)	12
4.2.6. Analisi delle Metriche e Colli di Bottiglia Strutturali	13
4.3. Architetture avanzate	15
4.3.1. Parallelismo Ibrido sul Nucleo Computazionale (Pipeline e Unroll su L2)	15
4.3.2. Analisi delle Metriche: Il Collo di Bottiglia delle Memorie	16
4.3.3. Risoluzione dei Conflitti di Memoria: Partizionamento Incrementale degli Array	19
4.3.4. Sovradimensionamento della Banda di Memoria (Unroll 2, Cyclic 4)	25
4.3.5. Analisi delle Metriche: Inefficacia del Sovradimensionamento	26
4.4. Architettura con "manual unroll"	29
4.4.1. Ristrutturazione Algoritmica: Srotolamento Manuale (Partial Sums)	29
4.4.2. Analisi delle Metriche: La Rottura delle Dipendenze Logiche	30
5. Sintesi dei Risultati e Conclusioni	36
5.1. Analisi conclusive	36
5.2. Conclusioni	37

1. Introduzione e Analisi degli Obiettivi

1.1. Descrizione del Progetto

Il Progetto 2 si basa sull'implementazione e l'ottimizzazione in High-Level Synthesis (HLS) dell'algoritmo di Moltiplicazione Matrice-Vettore Sparsa (SpMV). Questo algoritmo moltiplica una matrice contenente molti elementi pari a zero per un vettore, sfruttando un formato di compressione per evitare calcoli inutili. L'algoritmo base utilizza la codifica Compressed Row Storage (CRS) ed è caratterizzato da due cicli innestati: un ciclo esterno L1 che scorre le righe e un ciclo interno L2 che calcola i prodotti e le somme per gli elementi non nulli (NNZ) in ciascuna riga.

1.2. Sfida principale (loop a iterazioni variabile)

Il primo ostacolo nell'ottimizzazione dell'SpMV è che il numero di esecuzioni del ciclo interno L2 non è fisso, ma è dipendente dai dati in ingresso (varia in base a quanti elementi non nulli ci sono in una specifica riga). Di conseguenza, HLS non riesce a stimare la latenza e l'Initiation Interval (II) a tempo di sintesi, mostrando valori sconosciuti («?») nei report.

Questi limiti variabili influenzano pesantemente il comportamento delle direttive di ottimizzazione:

- **Pipeline e Unroll:** Se si tenta di applicare la direttiva di pipeline al ciclo esterno L1, HLS tenta automaticamente di effettuare l'*unroll* (srotolamento) completo del ciclo interno L2 per poter massimizzare il parallelismo.
- **Il fallimento della sintesi:** Tuttavia, l'unroll completo fallisce generando un warning (es. "Cannot unroll loop "L2" completely: variable loop bound"), proprio perché HLS non può generare hardware statico per un numero di iterazioni sconosciuto a tempo di compilazione.

1.3. Obiettivi

Il progetto analizza diverse soluzioni incrementali per affrontare questi colli di bottiglia. Gli obiettivi metodologici ed esplorativi del progetto sono:

- **Risoluzione diagnostica dei limiti variabili:** Utilizzare la direttiva `#pragma HLS loop_tripcount` per fornire al tool di sintesi i dati necessari (numero minimo, massimo e medio di iterazioni) per generare report prestazionali affidabili senza alterare il comportamento in sintesi logica.
- **Superamento dei colli di bottiglia della memoria:** Esplorare il partizionamento parziale e totale degli array per sbloccare i limiti hardware delle BRAM standard. Nello specifico, si sfrutterà un approccio *cyclic* per far sì che dati adiacenti si trovino in banchi RAM differenti, e un partizionamento *complete* per trasformare i vettori ad accesso indiretto (come `x`) in registri indipendenti.
- **Risoluzione della dipendenza di accumulo:** Risolvere la dipendenza *Read-After-Write* (Ricorrenza) sulla variabile di somma che costringe l'hardware a inserire uno stallo di attesa, bloccando l'Initiation Interval a 2.
- **Implementazione del Manual Loop Unrolling:** Modificare il codice C++ passando a uno srotolamento manuale per sfruttare appieno le diverse configurazioni di partizionamento degli array. Questo obiettivo chiave ristruttura l'algoritmo suddividendo il ciclo interno in un ciclo esterno a salti (su cui applicare la pipeline) e un ciclo interno a fattore fisso, isolando la dipendenza delle somme parziali tramite variabili temporanee e bypassando le limitazioni semantiche di HLS.

2. Metodologia di Esplorazione e Automazione del Workflow

2.1. Workflow automatizzato

Come ampiamente documentato nel primo progetto, il flusso di ottimizzazione ed esplorazione è stato interamente orchestrato tramite un **workflow automatizzato** basato su script Tcl. Questo ambiente di automazione consente di interfacciarsi direttamente con il tool di sintesi Vivado HLS, permettendo l'esecuzione in serie di molteplici configurazioni hardware e la successiva estrazione programmata dei dati dai report di sintesi e co-simulazione. L'adozione di questa metodologia strutturata riduce drasticamente i tempi di configurazione manuale tramite interfaccia grafica e previene potenziali discrepanze o errori di trascrizione nella raccolta dei dati prestazionali.

2.2. Approccio all'analisi

L'approccio metodologico adottato per l'ottimizzazione dell'algoritmo SpMV segue una strategia di **esplorazione incrementale dello spazio di progettazione** (*Design Space Exploration*). L'analisi si articola su un set di 11 casi studio progressivi, progettati per identificare e rimuovere sequenzialmente i colli di bottiglia fisici ed algoritmici che limitano il throughput del datapath.

La progressione logica dell'analisi prevede le seguenti fasi operative:

- **Identificazione e Diagnostica Baseline:** Inserimento iniziale della direttiva `#pragma HLS loop_tripcount` all'interno del ciclo interno L2. Questo passaggio è fondamentale per consentire ad HLS di stimare in modo deterministico la latenza e l'*Initiation Interval*, altrimenti contrassegnati da valori sconosciuti («?») nei report a causa dei limiti variabili dei cicli indotti dalla natura sparsa della matrice.
- **Isolamento dei Limiti Strutturali di Memoria:** Valutazione degli effetti del pipelining e dell'unrolling automatico su L2. L'incremento del parallelismo spaziale costringe a confrontarsi con i limiti fisici delle memorie BRAM standard (dotate al massimo di due porte), che non riescono a sostenere più letture contemporanee senza forzare la serializzazione degli accessi.
- **Risoluzione dei Conflitti di Banco tramite Partizionamento:** Applicazione della direttiva `ARRAY_PARTITION`. Si utilizza una strategia `cyclic` con fattore pari a F per i vettori ad accesso sequenziale (`values` e `col_idx`), garantendo che dati adiacenti risiedano in banchi fisici distinti, e un partizionamento di tipo `complete` per trasformare l'array ad accesso indiretto x in registri Flip-Flop indipendenti e privi di stalli.
- **Ristrutturazione Algoritmica del Codice:** Introduzione del **Manual Loop Unrolling** per sconfiggere il collo di bottiglia logico legato alla ricorsione dell'accumulatore. Separando il ciclo L2 in due sotto-cicli e isolando le somme su un array di registri temporanei indipendenti, si rompe la catena di dipendenze sequenziali "Read-After-Write", permettendo finalmente alla pipeline di raggiungere l'obiettivo prestazionale di $\mathbb{I} = 1$.

3. Metriche di Valutazione e Trade-off

Le metriche globali relative all'utilizzo delle risorse (LUT, FF, BRAM, DSP), alla latenza complessiva, alle prestazioni in frequenza e alle figure di merito (ADP ed EDP) rimangono invariate

rispetto a quelle definite e ampiamente discusse nella relazione del precedente progetto.¹

Tuttavia, considerata la natura dell'algoritmo SpMV, profondamente basato su cicli annidati con limiti di iterazione dipendenti dai dati, il framework di analisi è stato esteso per integrare un tracciamento dettagliato del comportamento dei singoli loop (come L1 ed L2). Le metriche specifiche relative ai loop, tabulate nei nuovi report, sono:

- **Trip count:** Indica il numero di iterazioni eseguite da un ciclo. Nell'implementazione base dell'SpMV, il ciclo interno presenta un *trip count* variabile, impedendo ad HLS di calcolare deterministicamente le latenze complessive senza l'ausilio di direttive diagnostiche esplicite.
- **Iteration Latency:** Misurata in cicli di clock e convertita in nanosecondi (utilizzando sia il *target clock* che l'*estimated clock*), rappresenta il tempo necessario per completare una singola iterazione del ciclo in esame.
- **Throughput:** Espresso in iterazioni al nanosecondo (iter/ns), quantifica la velocità con cui il datapath riesce a smaltire le operazioni del ciclo. È l'indicatore principale per valutare l'efficacia combinata del partizionamento delle memorie e delle tecniche di *unrolling*.
- **Pipelined e Initiation Interval (II):** Un flag booleano segnala se il ciclo è stato correttamente implementato in pipeline. Quando attivo, l'attenzione si sposta sull'*Initiation Interval*, ovvero il numero di cicli di clock necessari prima che il loop possa accettare i dati dell'iterazione successiva. L'obiettivo prestazionale ideale è raggiungere un $II = 1$, spesso ostacolato nell'SpMV dalle dipendenze di accumulo *Read-After-Write*.

4. Esplorazione dello Spazio di Design

4.1. Analisi dell'Architettura Baseline

4.1.1. Implementazione Logica di Base

Il codice sorgente dell'algoritmo SpMV implementa la moltiplicazione matrice-vettore sfruttando il formato di compressione **Compressed Row Storage (CRS)**, ideale per matrici sparse. I dati sono modellati tramite tipi `ap_int` ottimizzati e divisi in array dedicati: `values` per i valori non nulli, `col_idx` per gli indici di colonna corrispondenti, e `row_ptr` per tracciare l'inizio e la fine di ciascuna riga. La logica interna si articola su due cicli nidificati:

¹Si fa presente che, per alcune configurazioni architetturali intermedie o puramente esplorative, le metriche relative alla dissipazione di potenza e all'energia totale consumata non sono state valutate per snellire il processo di esplorazione.

```

...

// Iterate over rows
L1: for (int row = 0; row < NROWS; row++) {
    out_data_t sum = 0.0f;

    // Iterate over non-zero elements
    L2: for (int idx = row_ptr[row]; idx < row_ptr[row + 1]; idx++) {
        // --- MAC ---
        sum += values[idx] * x[col_idx[idx]];
    }

    y[row] = sum;
}

...

```

Codice 1: Cicli nidificati L1 (scorrimento righe) ed L2 (calcolo elementi non nulli).

All'interno dell'algoritmo si eseguono operazioni strettamente dipendenti:

- **Accesso indiretto alla memoria:** Per poter moltiplicare il valore estratto dalla matrice con il corrispondente elemento del vettore denso x , l'hardware deve prima leggere l'indice memorizzato in `col_idx` e usarlo come indirizzo per x ($x[\text{col_idx}[\text{idx}]]$).
- **Multiply-Accumulate (MAC):** I prodotti vengono sommati iterativamente nella variabile temporanea `sum`, che funge da accumulatore per la riga corrente.

Questa stesura iniziale, priva di direttive, forza il sintetizzatore HLS a implementare una macchina a stati puramente sequenziale, dove l'esecuzione dell'iterazione successiva di L2 deve attendere il completamento di tutti i colpi di clock necessari per i tre accessi in memoria e l'operazione aritmetica.

4.1.2. Testbench C/C++ Integrata

La robustezza formale dell'intera fase di esplorazione architetturale è garantita dal testbench `spmv_tb.cpp`. Questo script genera casualmente matrici e vettori e implementa un **golden model** (un simulatore matematico di riferimento, `golden_mvm`) eseguito in software per calcolare il vettore risultato atteso. Il risultato prodotto dall'IP hardware in co-simulazione viene confrontato iterativamente con l'uscita teorica (`sw_y`). Se il modulo mantiene l'equivalenza matematica, la routine restituisce uno stato di successo restituendo 0. Questo framework *self-checking* previene l'introduzione di errori logici durante l'applicazione aggressiva di direttive HLS.

4.1.3. Disabilitazione delle Ottimizzazioni Automatiche

Come configurato nel file `hls_config.cfg`, le ottimizzazioni implicite del tool sono state inibite per consentire uno studio metodologico passo-passo. L'impostazione dei parametri `syn.compile.pipeline_loops=0` e `syn.array_partition.throughput_driven=off` disabilita le euristiche di parallelismo automatico di Vivado HLS. Di conseguenza, questa *baseline* è una trasposizione *C-to-RTL* rigorosamente fedele al codice sorgente procedurale.

4.1.4. Analisi del Testbench VHDL e dei Constraint di Sistema

Terminato lo *stage* esplorativo in ambiente C++, il modulo generato viene validato fisicamente tramite il testbench RTL `project02_Spmv_00_baseline_tb.vhd`. A differenza di moduli con interfacce semplici, l'algoritmo SpMV richiede una notevole larghezza di banda verso la memoria.

Osservando il testbench VHDL, si nota come Vivado HLS abbia generato protocolli `ap_memory` indipendenti per ciascun array:

- Porte a singola lettura (`address0`, `ce0`, `q0`) per `values`, `x`, e `col_idx`.
- Una porta di scrittura dedicata per il vettore risultante `y` (`we0`, `d0`).
- Un'interfaccia **Dual-Port** per `row_ptr` (`address0/1`, `ce0/1`, `q0/1`), necessaria per leggere simultaneamente i limiti di inizio e fine iterazione del ciclo L2 (`row_ptr[row]` e `row_ptr[row + 1]`).

Inoltre, il target temporale è fissato a 10.0 ns tramite il parametro `clock=10ns` nel file di configurazione, guidando le stime di posizionamento logico del sintetizzatore.

4.1.5. Analisi delle Metriche di Valutazione e Limitazioni

Analisi preliminare

L'implementazione **baseline** originale soffre di un problema diagnostico critico: il numero di esecuzioni del ciclo interno L2 non è fisso, ma dipende dalla densità locale dei dati (limiti variabili). Di conseguenza, Vivado HLS non riesce a stimare la latenza totale a tempo di sintesi. Per analizzare rigorosamente questa soluzione di partenza, è stata iniettata esclusivamente la direttiva `#pragma HLS loop_tripcount` per fornire una media statistica delle iterazioni, sbloccando così la generazione di report affidabili senza alterare in alcun modo l'hardware sintetizzato.

Timing and Latency

Metric	At target clock	At estimated clock
Target clock period	10.000 ns	—
Estimated clock / $F_{\max}$	—	3.062 ns
Max achievable $F_{\max}$	—	326.580 MHz
Clock uncertainty	2.7 ns	
Latency min	1153cyc → 34570.0 ns (34.5700 μ s)	1153cyc → 10585.3 ns (10.5853 μ s)
Latency max	98689cyc → 34570.0 ns (34.5700 μ s)	98689cyc → 10585.3 ns (10.5853 μ s)
CoSim latency avg	3457cyc → 34570.0 ns (34.5700 μ s)	3457cyc → 10585.3 ns (10.5853 μ s)
CoSim total execution	3457cyc → 34570.0 ns (34.5700 μ s)	3457cyc → 10585.3 ns (10.5853 μ s)

Resource Utilisation

Resource	Used	Available	Utilisation
LUT	177	53200	0.33 %
FF	169	106400	0.16 %

BRAM	0	280	0.00 %
DSP	1	220	0.45 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			0.009461

Power and Energy

Metric	Value
Static power P_{static}	0.103 W
Dynamic power P_{dynamic}	0.001 W
Total power P_{total}	0.103 W
Energy/op $E_{\text{op}} = P_{\text{total}} \times \text{Latency}_{\text{max,est}}$	$1.090 \times 10^{-6} \text{ J}$
Total energy $E_{\text{total}} = P_{\text{total}} \times \text{Latency}_{\text{tot,est}}$	$1.090 \times 10^{-6} \text{ J}$

Trade-off Merits

Metric	Value
Area-Delay Product (ADP)	100.1478
Energy-Delay Product (EDP)	1.1541×10^{-2}

Loop Detail

Loop	Trip count	Iter. lat.	II	Pipelined
L1	128	771	—	no
L2	128	6	—	no

FSM Structure

Metric	Value
Number of FSM states	9
Critical path (max, any state)	3.060 ns

Analisi delle metriche

L'architettura generata rispecchia l'assenza totale di parallelismo. L'utilizzo delle risorse logiche è minimale, impiegando solamente 1 blocco DSP, 177 LUT e 169 FF sull'FPGA target, e non facendo uso di BRAM interne. Tuttavia, le prestazioni sono severamente limitate: il ciclo L2, responsabile del nucleo computazionale MAC, è totalmente sequenziale e richiede molteplici cicli di clock per iterazione a causa delle latenze intrinseche di accesso

in memoria in cascata ($\text{row_ptr} \rightarrow \text{col_idx} \rightarrow x$). Questo porta la latenza massima stimata per operazione a ben 98689 cicli di clock. Il throughput crolla drasticamente, riflettendosi su un tempo di esecuzione complessivo estremamente dilatato. Il periodo di clock stimato di 3.062 ns rispetta ampiamente il vincolo di 10.0 ns, limitando la potenza dinamica a soli 0.001 W, ma l'efficienza globale (EDP) risulta comunque compromessa dal ritardo computazionale.

Conclusioni

Il collo di bottiglia principale di questa implementazione risiede nelle limitazioni strutturali delle porte di memoria standard e nella rigida sequenzialità del datapath indotta dal ciclo L2. Il primo step ottimizzativo richiederà l'applicazione della direttiva di **Pipelining** sul ciclo interno per tentare di sovrapporre l'esecuzione delle moltiplicazioni. Questa mossa farà inevitabilmente emergere due nuovi ostacoli critici: le collisioni di accesso in memoria (che dovranno essere risolte tramite partizionamento *Cyclic* e *Complete* degli array) e le dipendenze di tipo *Read-After-Write* sull'accumulatore *sum*.

4.2. Architetture basi

4.2.1. Introduzione del Parallelismo Temporale (Loop Pipelining su L2)

Il primo passo ottimizzativo all'interno dello spazio esplorativo consiste nell'applicazione del **Loop Pipelining** sul nucleo computazionale dell'algoritmo, ovvero il ciclo interno L2. La direttiva `#pragma HLS pipeline II=1` istruisce il sintetizzatore a tentare di iniziare una nuova iterazione ad ogni colpo di clock, puntando a sovrapporre temporalmente l'esecuzione degli accessi in memoria e le operazioni di *Multiply-Accumulate* (MAC).

```
...

L1: for (int row = 0; row < NROWS; row++) {
    out_data_t sum = 0;

    L2: for (int idx = row_ptr[row]; idx < row_ptr[row + 1]; idx++) {
        #pragma HLS pipeline II=1
        sum += values[idx] * x[col_idx[idx]];
    }

    y[row] = sum;
}

...
```

Codice 2: Applicazione della direttiva di pipeline al ciclo interno L2.

Questa modifica non altera la logica algoritmica, validata nel *golden model* software, ma impone a Vivado HLS di rivoluzionare la schedulazione logica, passando da un'esecuzione puramente sequenziale a una struttura a stadi operativi concorrenti.

4.2.2. Analisi delle Metriche di Valutazione e Limitazioni

Analisi preliminare

L'inserimento della pipeline su L2 rappresenta il primo tentativo logico di spezzare la serialità dell'elaborazione per i dati non nulli. L'obiettivo imposto dal pragma è forzare il datapath a ingerire una nuova coppia di dati e produrre una somma parziale ogni singolo colpo di clock ($\mathbb{I} = 1$), mettendo sotto stress la disponibilità delle porte di memoria e la risoluzione delle dipendenze dei dati.

Timing and Latency

Metric	At target clock	At estimated clock
Target clock period	10.000 ns	—
Estimated clock / $F_{\max}$	—	4.226 ns
Max achievable $F_{\max}$	—	236.630 MHz
Clock uncertainty	2.7 ns	
Latency min	1409cyc $\rightarrow$ 15370.0 ns (15.3700 μ s)	1409cyc $\rightarrow$ 6495.4 ns (6.4954 μ s)
Latency max	17665cyc $\rightarrow$ 15370.0 ns (15.3700 μ s)	17665cyc $\rightarrow$ 6495.4 ns (6.4954 μ s)
CoSim latency avg	1537cyc $\rightarrow$ 15370.0 ns (15.3700 μ s)	1537cyc $\rightarrow$ 6495.4 ns (6.4954 μ s)
CoSim total execution	1537cyc $\rightarrow$ 15370.0 ns (15.3700 μ s)	1537cyc $\rightarrow$ 6495.4 ns (6.4954 μ s)

Resource Utilisation

Resource	Used	Available	Utilisation
LUT	231	53200	0.43 %
FF	258	106400	0.24 %
BRAM	0	280	0.00 %
DSP	1	220	0.45 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			0.011312

Power and Energy

Metric	Value
Static power P_{static}	0.103 W
Dynamic power P_{dynamic}	0.001 W
Total power P_{total}	0.103 W
Energy/op $E_{\text{op}} = P_{\text{total}} \times \text{Latency}_{\text{max,est}}$	6.690×10^{-7} J
Total energy $E_{\text{total}} = P_{\text{total}} \times \text{Latency}_{\text{tot,est}}$	6.690×10^{-7} J

Trade-off Merits

Metric	Value
Area-Delay Product (ADP)	73.4755
Energy-Delay Product (EDP)	4.3455×10^{-3}

Loop Detail

Loop	Trip count	Iter. lat.	II	Pipelined
L1	128	138	—	no
L2	128	6	1	yes

FSM Structure

Metric	Value
Number of FSM states	5
Critical path (max, any state)	4.200 ns

Analisi delle metriche

I dati estratti confermano il pieno successo della direttiva HLS: il ciclo L2 raggiunge effettivamente l'Initiation Interval ideale pari a 1. Questo risultato eccellente è reso possibile da due fattori architetturali intrinseci:

- **Assenza di collisioni in memoria:** Le letture necessarie all'iterazione coinvolgono tre array distinti (`values`, `col_idx` e `x`). Essendo mappati su istanze BRAM fisicamente separate, l'hardware può garantire un accesso simultaneo senza incorrere in stalli strutturali.
- **Risoluzione rapida della ricorrenza:** Operazione di **MAC** eseguita in un solo ciclo di clock grazie all'uso delle *DSP*.

A livello prestazionale, il throughput migliorato di L2 si riflette in un drastico abbattimento del tempo totale di esecuzione, che passa dagli oltre 10.5 us della baseline a soli 6.49 us. Come previsto per le strutture in pipeline, l'Area logica registra un lieve ma trascurabile incremento per quanto riguarda i Flip-Flop (saliti a 258), necessari per registrare i dati nei vari stage della pipeline. Il periodo di clock stimato rimane pienamente stabile a 4.226 ns, mantenendo altissimi i margini di tolleranza rispetto al vincolo di 10.0 ns.

Conclusioni

Sebbene la pipeline sul ciclo interno L2 abbia smaltito le operazioni a livello di singolo elemento non nullo nel modo più efficiente possibile per un'elaborazione scalare ($II = 1$), l'architettura è ancora parzialmente frenata. Persiste, infatti, il *loop overhead* causato dalle transizioni tra le varie righe nel ciclo sequenziale L1. Per spingere ulteriormente le

performance e incrementare la densità di calcolo temporale e spaziale, i prossimi step esplorativi dovranno necessariamente valutare l'applicazione del pipelining direttamente sul loop esterno L1 e, parallelamente, l'introduzione dello *srotolamento* del nucleo di calcolo, subordinato all'applicazione di severe tecniche di partizionamento per gli array in memoria. Per il momento continuiamo ad esplorare il comportamento del sintetizzatore sul loop L1.

4.2.3. Parallelismo Temporale sul Ciclo Esterno (Loop Pipelining su L1)

Il passo logico successivo nell'esplorazione del *design space* prevede lo spostamento della direttiva di pipelining dal ciclo interno a quello principale L1. L'intento teorico dell'inserimento di `#pragma HLS pipeline II=1` su L1 è forzare l'architettura a calcolare una riga intera della matrice per ogni colpo di clock, massimizzando il *throughput* globale.

```
...

L1: for (int row = 0; row < NROWS; row++) {
    #pragma HLS pipeline II=1
    out_data_t sum = 0;

    L2: for (int idx = row_ptr[row]; idx < row_ptr[row + 1]; idx++) {
        // --- MAC ---
        sum += values[idx] * x[col_idx[idx]];
    }

    y[row] = sum;
}

...
```

Codice 3: Applicazione della direttiva di pipeline al ciclo esterno L1.

Questa scelta architetturale impone un vincolo logico severo al sintetizzatore: per poter avviare una nuova iterazione di L1 in modo sovrapposto, tutti i sotto-cicli nidificati al suo interno (in questo caso L2) devono essere srotolati fisicamente in modo completo (*fully unrolled*).

4.2.4. Analisi delle Metriche e Fallimento Strutturale

Analisi preliminare

L'obiettivo di questa configurazione esplorativa è parallelizzare totalmente il nucleo computazionale a livello di riga. Tuttavia, forzare l'apertura del loop interno espone brutalmente il principale limite algoritmico della codifica CRS (Compressed Row Storage) valutata dal tool di sintesi HLS: la dipendenza dinamica dei limiti di iterazione.

Timing and Latency

Metric	At target clock	At estimated clock
Target clock period	10.000 ns	—
Estimated clock / $F_{\max}$	—	3.062 ns
Max achievable $F_{\max}$	—	326.580 MHz
Clock uncertainty	2.7 ns	

Latency min	1153cyc $\rightarrow$ 34570.0 ns (34.5700 μ s)	1153cyc $\rightarrow$ 10585.3 ns (10.5853 μ s)
Latency max	98689cyc $\rightarrow$ 34570.0 ns (34.5700 μ s)	98689cyc $\rightarrow$ 10585.3 ns (10.5853 μ s)
CoSim latency avg	3457cyc $\rightarrow$ 34570.0 ns (34.5700 μ s)	3457cyc $\rightarrow$ 10585.3 ns (10.5853 μ s)
CoSim total execution	3457cyc $\rightarrow$ 34570.0 ns (34.5700 μ s)	3457cyc $\rightarrow$ 10585.3 ns (10.5853 μ s)

Resource Utilisation

Resource	Used	Available	Utilisation
LUT	177	53200	0.33 %
FF	169	106400	0.16 %
BRAM	0	280	0.00 %
DSP	1	220	0.45 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			0.009461

Power and Energy

Metric	Value
Static power P_{static}	0.103 W
Dynamic power P_{dynamic}	0.001 W
Total power P_{total}	0.103 W
Energy/op $E_{\text{op}} = P_{\text{total}} \times \text{Latency}_{\text{max,est}}$	1.090×10^{-6} J
Total energy $E_{\text{total}} = P_{\text{total}} \times \text{Latency}_{\text{tot,est}}$	1.090×10^{-6} J

Trade-off Merits

Metric	Value
Area–Delay Product (ADP)	100.1478
Energy–Delay Product (EDP)	1.1541×10^{-2}

Loop Detail

Loop	Trip count	Iter. lat.	II	Pipelined
L1	128	771	—	no
L2	128	6	—	no

FSM Structure

Metric	Value
Number of FSM states	9
Critical path (max, any state)	3.060 ns

Analisi delle metriche

Le metriche estratte dai report indicano un palese collasso dell'ottimizzazione, con un ritorno matematico e prestazionale esatto ai valori dell'implementazione **baseline**. L'utilizzo delle risorse logiche è rimasto identico (1 DSP, 177 LUT e 169 FF), l'Initiation Interval del ciclo non è stato valutato e la latenza media di co-simulazione è regredita a 3457 cicli di clock (10.58 us).

Questa completa inefficacia logica è dovuta all'aborto del processo di *unrolling*. I limiti del ciclo L2 non sono costanti, ma dipendono dai valori assunti dinamicamente dall'array `row_ptr`. Non potendo calcolare deterministicamente a tempo di sintesi il quantitativo hardware (numero di moltiplicatori e sommatore) necessario per completare l'operazione in parallelo, Vivado HLS emette un *Warning* critico (impossibilità di srotolare completamente per limiti di ciclo variabili) e ignora a cascata la direttiva genitore di pipeline. Il datapath generato torna ad essere governato da una FSM puramente sequenziale.

Conclusioni

L'esito di questa fase dimostra l'impossibilità di scalare direttamente il parallelismo spaziale o temporale dall'esterno utilizzando il codice algoritmico nativo. Poiché le logiche euristiche automatiche del sintetizzatore si arrendono di fronte a limiti di ciclo non statici, l'unico modo per superare questo ostacolo è aggirare manualmente sulla struttura del codice. Nel prossimo passo verrà tentato di eseguire **parallelismo spaziale** per analizzare anche questa opzione.

4.2.5. Parallelismo Spaziale sul Ciclo Esterno (Loop Unroll Parziale su L1)

In seguito al fallimento del partizionamento temporale totale, la strategia di esplorazione si sposta sull'introduzione del parallelismo spaziale a livello di riga. L'applicazione della direttiva `#pragma HLS unroll factor=2` sul ciclo esterno L1 ha l'obiettivo di forzare l'istanziamento di logica hardware duplicata, permettendo teoricamente all'IP di elaborare due righe della matrice sparsa contemporaneamente.

```

...

L1: for (int row = 0; row < NROWS; row++) {
    #pragma HLS unroll factor=2
    out_data_t sum = 0;

    L2: for (int idx = row_ptr[row]; idx < row_ptr[row + 1]; idx++) {
        // --- MAC ---
        sum += values[idx] * x[col_idx[idx]];
    }

    y[row] = sum;
}

...

```

Codice 4: Applicazione della direttiva di srotolamento spaziale (fattore 2) al ciclo esterno L1.

A livello architetturale, questa scelta istruisce il sintetizzatore ad abbassare il *trip count* del loop esterno (da 128 a 64 iterazioni totali) e a replicare integralmente le logiche interne e le unità operative aritmetiche del datapath per gestire due esecuzioni parallele di L2.

4.2.6. Analisi delle Metriche e Colli di Bottiglia Strutturali

Analisi preliminare

Lo srotolamento parziale del ciclo esterno rappresenta un tentativo di *Trade-off* tra area e velocità: raddoppiare le risorse disponibili per dimezzare i tempi operativi. Tuttavia, senza un'adeguata architettura di memoria sottostante, le unità di calcolo parallele rischiano di rimanere in stallo per mancanza di dati in ingresso (fenomeno di *data starvation*).

Timing and Latency

Metric	At target clock	At estimated clock
Target clock period	10.000 ns	—
Estimated clock / $F_{\max}$	—	3.062 ns
Max achievable $F_{\max}$	—	326.580 MHz
Clock uncertainty	2.7 ns	
Latency min	1089cyc → 33930.0 ns (33.9300 μ s)	1089cyc → 10389.4 ns (10.3894 μ s)
Latency max	98625cyc → 33930.0 ns (33.9300 μ s)	98625cyc → 10389.4 ns (10.3894 μ s)
CoSim latency avg	3393cyc → 33930.0 ns (33.9300 μ s)	3393cyc → 10389.4 ns (10.3894 μ s)
CoSim total execution	3393cyc → 33930.0 ns (33.9300 μ s)	3393cyc → 10389.4 ns (10.3894 μ s)

Resource Utilisation

Resource	Used	Available	Utilisation
LUT	375	53200	0.70 %
FF	310	106400	0.29 %
BRAM	0	280	0.00 %
DSP	2	220	0.91 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			0.019053

Power and Energy

Metric	Value
Static power P_{static}	0.103 W
Dynamic power P_{dynamic}	0.002 W
Total power P_{total}	0.104 W
Energy/op $E_{\text{op}} = P_{\text{total}} \times \text{Latency}_{\text{max,est}}$	1.080×10^{-6} J
Total energy $E_{\text{total}} = P_{\text{total}} \times \text{Latency}_{\text{tot,est}}$	1.080×10^{-6} J

Trade-off Merits

Metric	Value
Area-Delay Product (ADP)	197.9486
Energy-Delay Product (EDP)	1.1226×10^{-2}

Loop Detail

Loop	Trip count	Iter. lat.	II	Pipelined
L1	64	1541	—	no
L2	128	6	—	no
L2	128	6	—	no

FSM Structure

Metric	Value
Number of FSM states	16
Critical path (max, any state)	3.060 ns

Analisi delle metriche

L'analisi dei report logici e temporali conferma l'avvenuta sintesi della direttiva spaziale. Il raddoppio strutturale è evidente nel consumo d'Area: vengono inferiti 2 blocchi DSP, 375 LUT e 310 FF, ovvero esattamente il doppio rispetto alla controparte sequenziale o in pipeline.

Tuttavia, il reale impatto prestazionale è fortemente deludente. Il tempo totale misurato in co-simulazione scende solamente da 3457 a 3393 cicli di clock (10.38 us). Questa inefficienza estrema è causata da un profondo *Structural Hazard*. Sebbene l'hardware disponga di due *Multiply-Accumulate* indipendenti per calcolare le righe simultaneamente, entrambe le istanze necessitano di accedere alle strutture vettoriali compresse (`row_ptr`, `col_idx` e `values`). Poiché gli array non hanno subito partizionamento logico e si appoggiano a singole entità BRAM standard (limitate per natura a un massimo di due porte fisiche), non esiste banda passante sufficiente per sostenere l'estrazione parallela di tutti i dati richiesti. Di conseguenza, lo scheduler di Vivado HLS interviene inserendo stalli forzati e serializzando nuovamente gli accessi in lettura, annullando nei fatti l'intero guadagno temporale dello *srotolamento*.

Conclusioni

Questa variante esplorativa certifica inequivocabilmente che ottimizzare il livello esterno dell'algoritmo SpMV in assenza di un riadattamento della memoria produce risultati asimmetrici, peggiorando drasticamente il rapporto *Area-Delay* (ADP). L'azione logica deve necessariamente tornare sul nucleo vitale dell'elaborazione: il ciclo interno L2. Le prossime iterazioni si focalizzeranno sull'unione ibrida di parallelismo temporale (Pipeline) e spaziale (Unroll) direttamente su L2, accompagnate da prime e indispensabili tecniche di partizionamento per arginare i conflitti architetturali.

4.3. Architetture avanzate

4.3.1. Parallelismo Ibrido sul Nucleo Computazionale (Pipeline e Unroll su L2)

Constatata l'inefficacia dello srotolamento spaziale applicato unicamente al ciclo esterno L1, l'indagine architettonica si concentra nuovamente sul nucleo computazionale dell'algoritmo. L'obiettivo di questa fase esplorativa è fondere il parallelismo temporale con quello spaziale, agendo direttamente sul ciclo interno L2. Vengono applicate congiuntamente le direttive `#pragma HLS pipeline II=1` e `#pragma HLS unroll factor=2`.


```

...

L1: for (int row = 0; row < NROWS; row++) {
    out_data_t sum = 0;

    L2: for (int idx = row_ptr[row]; idx < row_ptr[row + 1]; idx++) {
        #pragma HLS pipeline II=1
        #pragma HLS unroll factor=2
        // --- MAC ---
        sum += values[idx] * x[col_idx[idx]];
    }

    y[row] = sum;
}

...

```

Codice 5: Applicazione congiunta di pipeline e srotolamento parziale sul ciclo L2.

Questa architettura tenta di istanziare e orchestrare due unità logiche indipendenti per prelevare, moltiplicare e accumulare due elementi non nulli simultaneamente ad ogni colpo di clock.

4.3.2. Analisi delle Metriche: Il Collo di Bottiglia delle Memorie

Analisi preliminare

Il confronto viene effettuato prendendo come riferimento la soluzione `01_pipeline-L2`, la quale incarnava il miglior compromesso prestazionale provvisorio avendo garantito un target ideale di $\mathbb{I} = 1$ in elaborazione puramente sequenziale. L'aspettativa teorica per questa nuova variante ibrida prevede un dimezzamento dei tempi di esecuzione a fronte di un fisiologico raddoppio dell'occupazione logica.

Timing

Metric	01_pipeline-L2	04_pipeline-unroll-2-L2	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.226 ns	4.978 ns	▲ 17.79 %
Max achievable $F_{\max}$	236.630 MHz	200.880 MHz	▼ 15.11 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	01_pipeline-L2	04_pipeline-unroll-2-L2	Δ
Latency max @ target clock	17665cyc → 15370.0 ns (15.3700 μ s)	17793cyc → 17930.0 ns (17.9300 μ s)	▲ 16.66 %

Latency max @ estimated clock	17665cyc $\rightarrow$ 6495.4 ns (6.4954 μ s)	17793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	$\blacktriangle$ 37.41 %
CoSim total exec. @ target clock	1537cyc $\rightarrow$ 15370.0 ns (15.3700 μ s)	1793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	$\blacktriangle$ 16.66 %
CoSim total exec. @ estimated clock	1537cyc $\rightarrow$ 6495.4 ns (6.4954 μ s)	1793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	$\blacktriangle$ 37.41 %

Resource Utilisation

Metric	01_pipeline-L2	04_pipeline-unroll-2-L2	Δ
LUT	231 (0.43 %)	392 (0.74 %)	$\blacktriangle$ 69.70 %
FF	258 (0.24 %)	406 (0.38 %)	$\blacktriangle$ 57.36 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	1 (0.45 %)	2 (0.91 %)	$\blacktriangle$ 100.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			$\blacktriangle$ 79.23 %

Power and Energy

Metric	01_pipeline-L2	04_pipeline-unroll-2-L2	Δ
Static power P_{static}	0.103 W	0.103 W	= 0.00 %
Dynamic power P_{dynamic}	0.001 W	0.002 W	$\blacktriangle$ 100.00 %
Total power P_{total}	0.103 W	0.104 W	$\blacktriangle$ 0.97 %
Energy/op E_{op}	6.690×10^{-7} J	9.283×10^{-7} J	$\blacktriangle$ 38.75 %
Total energy E_{total}	6.690×10^{-7} J	9.283×10^{-7} J	$\blacktriangle$ 38.75 %

Figures of Merit

Metric	01_pipeline-L2	04_pipeline-unroll-2-L2	Δ
Area-Delay Product (ADP)	73.4755	180.9656	$\blacktriangle$ 146.29 %
Energy-Delay Product (EDP)	4.3455×10^{-3}	8.2852×10^{-3}	$\blacktriangle$ 90.66 %

Loop Comparison

Loop	Trip count	Iter. latency (cyc)	II
------	------------	---------------------	----

	01	04	Δ	01	04	Δ	01	04	Δ
L1	128	128	= 0.00 %	138	139	▲ 0.72 %	—	—	—
L2	128	64	▼ 50.00 %	6	7	▲ 16.67 %	1	2	▲ 100.00 %

FSM Structure

Metric	01_pipeline-L2	04_pipeline-un-roll-2-L2	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	4.200 ns	2.770 ns	▼ 34.05 %

Analisi delle metriche

Le metriche comparative evidenziano un marcato degrado delle prestazioni temporali, smentendo le proiezioni algoritmiche ottimali. Il conteggio dei cicli totali estratti in co-simulazione subisce un deterioramento, innalzandosi da 1537 a 1793 cicli, spingendo il tempo di esecuzione complessivo stimato fino a 8.92 us.

Il fallimento architetturale si annida nella discrepanza tra le unità di calcolo e l'infrastruttura di alimentazione dei dati. Sebbene il parametro *trip count* si sia fisicamente dimezzato come indotto dall'unroll con fattore 2, Vivado HLS rileva profonde contese strutturali sulle porte fisiche di memoria, essendo costretto a far innalzare l'*Initiation Interval* a $\mathbb{I} = 2$. Il datapath, per elaborare due istruzioni parallele, richiede il fetch simultaneo di due elementi contigui dall'array *values* e altrettanti indici dall'array *col_idx*. Operando su singole interfacce BRAM prive di partizionamento, il cui limite fisico insuperabile è di due porte di lettura totali, lo scheduler interviene inserendo inevitabili stalli per serializzare l'estrazione. Sul fronte dell'impronta hardware, l'Area rispecchia pedissequamente la sintesi attesa: si registra un raddoppio dei moduli DSP (da 1 a 2), coerentemente supportato da un aumento dei registri di instradamento (FF) e blocchi logici (LUT).

Conclusioni

La combinazione di Pipeline e Unroll sul medesimo ciclo computazionale fallisce senza una redistribuzione speculare del carico sulla memoria. L'insufficienza di banda passante (*memory bandwidth*) strozza l'alimentazione del datapath in parallelo, rendendo le risorse logiche duplicate inattive per la metà del tempo. Per ripristinare il target di $\mathbb{I} = 1$ garantendo la produttività parallela, i passaggi successivi impongono la stesura di logiche rigorose di memory partitioning tramite l'uso mirato della direttiva `ARRAY_PARTITION`.

4.3.3. Risoluzione dei Conflitti di Memoria: Partizionamento Incrementale degli Array

Per sbloccare il parallelismo spaziale dettato dallo srotolamento del ciclo L2 (fattore 2), è imperativo fornire all'hardware una larghezza di banda verso la memoria sufficiente ad alimentare le due unità logiche simultaneamente. Nelle tre architetture esplorate in questa fase, sono state iniettate direttive di partizionamento (`#pragma HLS array_partition`) in modo incrementale per distribuire gli elementi vettoriali su banchi fisici indipendenti senza alterare il funzionamento algoritmico del *golden model*.

```
...  
  
// V1: Partizionamento di 'values'  
#pragma HLS array_partition variable=values cyclic factor=2 dim=1  
  
// V2: Aggiunta del partizionamento di 'col_idx'  
#pragma HLS array_partition variable=col_idx cyclic factor=2 dim=1  
  
// V3: Aggiunta del partizionamento totale di 'x'  
#pragma HLS array_partition variable=x complete dim=1  
  
...
```

Codice 6: Applicazione incrementale delle direttive di partizionamento in accoppiata a Pipeline e Unroll.

Analisi preliminare

Questa sequenza esplorativa mira ad annullare i contenziosi strutturali sulle porte BRAM rilevati con il fallimento dell'ibridazione.

- La **V1** (Soluzione 05) applica un partizionamento *cyclic* con fattore 2 all'array `values`, dividendo fisicamente i dati in due memorie distinte con accessi interleaved.
- La **V2** (Soluzione 06) estende la direttiva *cyclic* anche all'array degli indici `col_idx`, garantendone l'estrazione in tandem.
- La **V3** (Soluzione 07) interviene in maniera aggressiva sul vettore denso `x`: essendo questo ad accesso indiretto (guidato dal valore estratto da `col_idx`), un partizionamento ciclico non assicurerebbe la risoluzione delle collisioni causate da letture casuali. Si opta quindi per un partizionamento *complete*, che dissolve l'intera struttura di memoria ramificandola in singoli registri Flip-Flop.

I confronti tabellari sottostanti misurano progressivamente l'impatto di queste topologie di interconnessione prendendo sempre come traguardo la Soluzione 01, l'unica ad aver finora garantito un Initiation Interval ideale di 1.

Timing

Metric	04_pipeline-unroll-2-L2	05_part-v1	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.978 ns	4.978 ns	= 0.00 %
Max achievable $F_{\max}$	200.880 MHz	200.880 MHz	= 0.00 %

Clock uncertainty	2.7 ns	2.7 ns	—
-------------------	--------	--------	---

Latency and Throughput

Metric	04_pipeline-unroll-2-L2	05_part-v1	Δ
Latency max @ target clock	17793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	17793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	= 0.00 %
Latency max @ estimated clock	17793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	17793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	= 0.00 %
CoSim total exec. @ target clock	1793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	1793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	= 0.00 %
CoSim total exec. @ estimated clock	1793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	1793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	= 0.00 %

Resource Utilisation

Metric	04_pipeline-unroll-2-L2	05_part-v1	Δ
LUT	392 (0.74 %)	408 (0.77 %)	$\blacktriangle$ 4.08 %
FF	406 (0.38 %)	384 (0.36 %)	$\blacktriangledown$ 5.42 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	2 (0.91 %)	2 (0.91 %)	= 0.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			$\blacktriangle$ 0.46 %

Power and Energy

Metric	04_pipeline-unroll-2-L2	05_part-v1	Δ
Static power P_{static}	0.103 W	— W	—
Dynamic power P_{dynamic}	0.002 W	— W	—
Total power P_{total}	0.104 W	— W	—
Energy/op E_{op}	9.283×10^{-7} J	— J	—
Total energy E_{total}	9.283×10^{-7} J	— J	—

Figures of Merit

Metric		04_pipeline-unroll-2-L2	05_part-v1	Δ
Area-Delay (ADP)	Product	180.9656	181.8046	▲ 0.46 %
Energy-Delay (EDP)	Product	8.2852×10^{-3}	—	—

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	04	05	Δ	04	05	Δ	04	05	Δ
L1	128	128	= 0.00 %	139	139	= 0.00 %	—	—	—
L2	64	64	= 0.00 %	7	7	= 0.00 %	2	2	= 0.00 %

FSM Structure

Metric	04_pipeline-unroll-2-L2	05_part-v1	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	2.770 ns	2.770 ns	= 0.00 %

Timing

Metric	04_pipeline-unroll-2-L2	06_part-v2	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.978 ns	4.978 ns	= 0.00 %
Max achievable $F_{\max}$	200.880 MHz	200.880 MHz	= 0.00 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	04_pipeline-unroll-2-L2	06_part-v2	Δ
--------	-------------------------	------------	----------

Latency max @ target clock	17793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	17793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	= 0.00 %
Latency max @ estimated clock	17793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	17793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	= 0.00 %
CoSim total exec. @ target clock	1793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	1793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	= 0.00 %
CoSim total exec. @ estimated clock	1793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	1793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	= 0.00 %

Resource Utilisation

Metric	04_pipeline-unroll-2-L2	06_part-v2	Δ
LUT	392 (0.74 %)	422 (0.79 %)	▲ 7.65 %
FF	406 (0.38 %)	384 (0.36 %)	▼ 5.42 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	2 (0.91 %)	2 (0.91 %)	= 0.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			▲ 1.76 %

Power and Energy

Metric	04_pipeline-unroll-2-L2	06_part-v2	Δ
Static power P_{static}	0.103 W	— W	—
Dynamic power P_{dynamic}	0.002 W	— W	—
Total power P_{total}	0.104 W	— W	—
Energy/op E_{op}	9.283×10^{-7} J	— J	—
Total energy E_{total}	9.283×10^{-7} J	— J	—

Figures of Merit

Metric	04_pipeline-unroll-2-L2	06_part-v2	Δ
Area-Delay Product (ADP)	180.9656	184.1520	▲ 1.76 %
Energy-Delay Product (EDP)	8.2852×10^{-3}	—	—

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	04	06	Δ	04	06	Δ	04	06	Δ
L1	128	128	= 0.00 %	139	139	= 0.00 %	—	—	—
L2	64	64	= 0.00 %	7	7	= 0.00 %	2	2	= 0.00 %

FSM Structure

Metric	04_pipeline-un- roll-2-L2	06_part-v2	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	2.770 ns	2.770 ns	= 0.00 %

Timing

Metric	04_pipeline-un- roll-2-L2e	07_part-v3	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.978 ns	4.978 ns	= 0.00 %
Max achievable $F_{\max}$	200.880 MHz	200.880 MHz	= 0.00 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	04_pipeline-un- roll-2-L2e	07_part-v3	Δ
Latency max @ target clock	17793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	17665cyc $\rightarrow$ 16650.0 ns (16.6500 μ s)	▼ 7.14 %
Latency max @ estimated clock	17793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	17665cyc $\rightarrow$ 8288.4 ns (8.2884 μ s)	▼ 7.14 %
CoSim total exec. @ target clock	1793cyc $\rightarrow$ 17930.0 ns (17.9300 μ s)	1665cyc $\rightarrow$ 16650.0 ns (16.6500 μ s)	▼ 7.14 %

CoSim total exec. @ estimated clock	1793cyc $\rightarrow$ 8925.6 ns (8.9256 μ s)	1665cyc $\rightarrow$ 8288.4 ns (8.2884 μ s)	$\blacktriangledown$ 7.14 %
-------------------------------------	---	---	------------------------------------

Resource Utilisation

Metric	04_pipeline-unroll-2-L2e	07_part-v3	Δ
LUT	392 (0.74 %)	1747 (3.28 %)	$\blacktriangle$ 345.66 %
FF	406 (0.38 %)	1654 (1.55 %)	$\blacktriangle$ 307.39 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	2 (0.91 %)	2 (0.91 %)	= 0.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			$\blacktriangle$ 183.47 %

Power and Energy

Metric	04_pipeline-unroll-2-L2e	07_part-v3	Δ
Static power P_{static}	0.103 W	— W	—
Dynamic power P_{dynamic}	0.002 W	— W	—
Total power P_{total}	0.104 W	— W	—
Energy/op E_{op}	9.283×10^{-7} J	— J	—
Total energy E_{total}	9.283×10^{-7} J	— J	—

Figures of Merit

Metric	04_pipeline-unroll-2-L2e	07_part-v3	Δ
Area-Delay Product (ADP)	180.9656	476.3658	$\blacktriangle$ 163.24 %
Energy-Delay Product (EDP)	8.2852×10^{-3}	—	—

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	04	07	Δ	04	07	Δ	04	07	Δ
L1	128	128	= 0.00 %	139	138	$\blacktriangledown$ 0.72 %	—	—	—

L2	64	64	= 0.00 %	7	6	▼ 14.29 %	2	2	= 0.00 %
----	----	----	----------------	---	---	--------------	---	---	----------------

FSM Structure

Metric	04_pipeline-unroll-2-L2e	07_part-v3	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	2.770 ns	2.770 ns	= 0.00 %

Conclusioni

L'analisi incrociata dei tre report svela l'evoluzione e i limiti implacabili dello srotolamento automatico. Nelle prime due iterazioni (**V1** e **V2**), l'assenza del partizionamento totale sul vettore x mantiene l'hardware strozzato. Di conseguenza, il ciclo è costretto a iterare con $\mathbb{I} = 2$, accumulando un ritardo computazionale paradossalmente superiore rispetto alla controparte puramente in pipeline (1793 cicli contro 1537).

Con la configurazione finale **V3**, i conflitti fisici verso i banchi di memoria vengono letteralmente eliminati: la totale trasformazione di x in celle di memoria discrete si riflette nell'impennata di risorse necessarie al routing, portando i Flip-Flop a quota 1656 e le LUT a quota 1747. Nonostante l'hardware goda finalmente di un flusso di alimentazione dei dati illimitato, l'Initiation Interval si ostina a rimanere a 2, facendo scendere timidamente il totale a soli 1665 cicli.

L'ostacolo individuato non è più di natura strutturale, bensì logica e semantica: lo srotolamento automatico applicato a un loop dipendente dai dati obbliga Vivado HLS a preservare ad ogni costo l'ordine rigoroso delle somme sull'accumulatore `sum`, creando una catena invalicabile di dipendenze di tipo *Read-After-Write*. L'unica e decisiva manovra per bypassare le restrizioni del linguaggio generato e ripristinare il traguardo di $\mathbb{I} = 1$ su hardware duplicato richiede la ristrutturazione algoritmica del C++ tramite il **Manual Loop Unrolling**, tecnica che separerà forzatamente l'aggiornamento dell'accumulatore distribuendolo su variabili intermedie indipendenti.

Prima di verificare ciò viene tentato di spingere ulteriormente il partizionamento per verificare se ciò lascia invariato il risultato.

4.3.4. Sovradimensionamento della Banda di Memoria (Unroll 2, Cyclic 4)

L'indagine procede esplorando gli effetti di un disallineamento volontario tra il parallelismo spaziale e la banda passante della memoria. Mantenendo lo srotolamento del ciclo L2 a un fattore di 2, si innalza il partizionamento ciclico degli array vettoriali `values` e `col_idx` a un fattore di 4.

```

...

#pragma HLS array_partition variable=values cyclic factor=4 dim=1
#pragma HLS array_partition variable=col_idx cyclic factor=4 dim=1
#pragma HLS array_partition variable=x complete dim=1

L1: for (int row = 0; row < NROWS; row++) {
    out_data_t sum = 0;

    L2: for (int idx = row_ptr[row]; idx < row_ptr[row + 1]; idx++) {
        #pragma HLS pipeline II=1
        #pragma HLS unroll factor=2
        // --- MAC ---
        sum += values[idx] * x[col_idx[idx]];
    }

    y[row] = sum;
}

...

```

Codice 7: Applicazione di un partizionamento ciclico con fattore 4 a fronte di uno srotolamento con fattore 2 sul ciclo L2.

L'obiettivo teorico di questa topologia asimmetrica è verificare empiricamente se una sovrabbondanza di porte fisiche in lettura possa sbloccare le limitazioni logiche dello scheduler di Vivado HLS, tentando di abbattere l'Initiation Interval rimasto ancorato a 2 nella soluzione simmetrica precedente.

4.3.5. Analisi delle Metriche: Inefficacia del Sovradimensionamento

Analisi preliminare

Il confronto oppone la configurazione simmetrica ottimale (Unroll 2, Cyclic 2) alla nuova variante sovradimensionata (Unroll 2, Cyclic 4). L'analisi mira a quantificare l'impatto di un'infrastruttura di memoria sproporzionata rispetto alle reali capacità di assorbimento in ingresso del datapath generato.

Timing

Metric	07_part-v3	08_cyclic-4	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.978 ns	4.978 ns	= 0.00 %
Max achievable $F_{\max}$	200.880 MHz	200.880 MHz	= 0.00 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	07_part-v3	08_cyclic-4	Δ
--------	------------	-------------	----------

Latency max @ target clock	17665cyc → 16650.0 ns (16.6500 μ s)	17665cyc → 16650.0 ns (16.6500 μ s)	= 0.00 %
Latency max @ estimated clock	17665cyc → 8288.4 ns (8.2884 μ s)	17665cyc → 8288.4 ns (8.2884 μ s)	= 0.00 %
CoSim total exec. @ target clock	1665cyc → 16650.0 ns (16.6500 μ s)	1665cyc → 16650.0 ns (16.6500 μ s)	= 0.00 %
CoSim total exec. @ estimated clock	1665cyc → 8288.4 ns (8.2884 μ s)	1665cyc → 8288.4 ns (8.2884 μ s)	= 0.00 %

Resource Utilisation

Metric	07_part-v3	08_cyclic-4	Δ
LUT	1747 (3.28 %)	1785 (3.36 %)	▲ 2.18 %
FF	1654 (1.55 %)	1655 (1.56 %)	▲ 0.06 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	2 (0.91 %)	2 (0.91 %)	= 0.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			▲ 1.26 %

Power and Energy

Metric	07_part-v3	08_cyclic-4	Δ
Static power P_{static}	— W	— W	—
Dynamic power P_{dynamic}	— W	— W	—
Total power P_{total}	— W	— W	—
Energy/op E_{op}	— J	— J	—
Total energy E_{total}	— J	— J	—

Figures of Merit

Metric	07_part-v3	08_cyclic-4	Δ
Area–Delay Product (ADP)	476.3658	482.3666	▲ 1.26 %
Energy–Delay Product (EDP)	—	—	—

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	07	08	Δ	07	08	Δ	07	08	Δ
L1	128	128	= 0.00 %	138	138	= 0.00 %	—	—	—
L2	64	64	= 0.00 %	6	6	= 0.00 %	2	2	= 0.00 %

FSM Structure

Metric	07_part-v3	08_cyclic-4	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	2.770 ns	2.770 ns	= 0.00 %

Analisi delle metriche

Le metriche estratte certificano che un sovradimensionamento della larghezza di banda non apporta alcun beneficio prestazionale, confermando che il collo di bottiglia strutturale verso le porte BRAM era già stato pienamente risolto nella variante precedente. L'Initiation Interval rimane inesorabilmente bloccato a $II = 2$ e i cicli totali misurati in co-simulazione si stabilizzano allo stesso esatto valore di 1665 cicli.

Tuttavia, l'impronta hardware subisce un lieve ma tangibile peggioramento: pur mantenendo invariato l'impiego dei 2 DSP (essendo l'unroll limitato a fattore 2), le risorse logiche subiscono una variazione al rialzo, con le LUT che salgono da 1747 a 1785. Questo incremento d'area parassita è fisiologico ed è imputabile alla logica di moltiplicazione (*Muxing*) supplementare che Vivado HLS deve generare per gestire l'instradamento incrociato da 4 banchi di memoria fisici indipendenti verso le sole 2 unità operative attive. L'hardware possiede la capacità di estrarre 4 dati per ciclo, ma il datapath ne richiede sempre e solo 2, lasciando metà della banda sprecata.

Conclusioni

L'esperimento cristallizza un principio fondamentale della progettazione HLS: il fattore di partizionamento delle memorie (specialmente con approccio *cyclic*) deve essere sempre simmetrico e strettamente allineato al fattore di srotolamento del ciclo. Fornire risorse periferiche eccedenti aumenta il costo del routing logico senza dissolvere la limitazione dominante dell'architettura: la dipendenza semantica (Data Hazard) legata all'accumulatore *sum*. Essendo esaurite le potenzialità logiche dell'unroll automatico, le iterazioni esplorative successive impongono il passaggio obbligato al **Manual Loop Unrolling** per spezzare in

modo definitivo l'albero delle dipendenze e sbloccare finalmente il parallelismo spaziale a $\mathbb{I} = 1$.

4.4. Architettura con “manual unroll”

4.4.1. Ristrutturazione Algoritmica: Srotolamento Manuale (Partial Sums)

Arrivati al limite semantico del linguaggio C++ generato da HLS, dove le direttive automatiche non riescono a dissolvere la rigida dipendenza algoritmica legata alla variabile di accumulo `sum` in presenza di cicli a limite variabile, si rende necessaria una profonda ristrutturazione del codice. La tecnica dello **Srotolamento Manuale** (*Manual Loop Unrolling*) scinde l'iterazione interna in due domini separati e introduce un array intermedio `partial_sums` per distribuire temporaneamente il calcolo su memorie parallele.

```
...

out_data_t partial_sums[F*2] = {0};
#pragma HLS array_partition variable=partial_sums type=complete

// L2_1: Avanza a salti di F*2 (ingestione parallela)
L2_1: for (int k = start; k < end; k += F*2) {
    /* MIN: at lest 1 non-zero element
    /* MAX: 128 non-zero elements (entire row), now F*2 in parallel
    /* AVG: 4 non-zero elements (typically 512 non-zero elements i total,
512/128 = 4 per row), now F*2 in parallel
    #pragma HLS loop_tripcount min=1 max=128/(F*2) avg=4/(F*2)
    #pragma HLS pipeline II=1

    // L2_2: Srotolamento fisico per operazioni MAC indipendenti
    L2_2: for (int i = 0; i < F*2; i++) {
        #pragma HLS unroll
        int idx = k + i;

        if (idx < end) {
            partial_sums[i] += values[idx] * x[col_idx[idx]];
        }
    }
}
```

Codice 8: Implementazione del *Manual Unrolling* con array di somme parziali e tripcount aggiornato al reale carico per iterazione.

Oltre a frammentare la dipendenza *Read-After-Write* (Data Hazard), questo approccio introduce un'ottimizzazione ingegneristica basata sull'architettura fisica del dispositivo FPGA. I blocchi BRAM interni possiedono nativamente una topologia **True Dual-Port**, che garantisce fino a due letture indipendenti per colpo di clock sulla medesima memoria. Sfruttando questa proprietà, e applicando un fattore di partizionamento *cyclic* pari a 2, HLS divide i vettori in soli 2 banchi fisici. Tuttavia, sommando le interfacce, il sistema espone ben 4 porte logiche di lettura in parallelo (2 BRAM separate moltiplicato per 2 porte logiche per BRAM). Di conseguenza, pur mantenendo un modesto fattore di partizionamento spaziale, il datapath può essere logicamente srotolato con un fattore di 4 ($F*2$, assumendo $F=2$), smaltendo quattro campioni contigui alla

volta senza incorrere in stalli ed evitando il dispendioso costo di instradamento visto nei passati tentativi con partizionamento a quattro vie.

4.4.2. Analisi delle Metriche: La Rottura delle Dipendenze Logiche

Analisi preliminare

Il primo confronto oppone l'implementazione a srotolamento manuale (Soluzione 09) al primissimo tentativo ingenuo di srotolamento automatico privo di partizionamento (Soluzione 04). L'obiettivo è misurare il salto prestazionale globale ottenuto passando da un'architettura strozzata logicamente e strutturalmente, a una topologia ottimizzata in ogni suo aspetto.

Timing

Metric	04_auto-unroll	09_manual-unroll	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.978 ns	5.128 ns	▲ 3.01 %
Max achievable $F_{\max}$	200.880 MHz	195.010 MHz	▼ 2.92 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	04_auto-unroll	09_manual-unroll	Δ
Latency max @ target clock	17793cyc → 17930.0 ns (17.9300 μ s)	5377cyc → 12810.0 ns (12.8100 μ s)	▼ 28.56 %
Latency max @ estimated clock	17793cyc → 8925.6 ns (8.9256 μ s)	5377cyc → 6569.0 ns (6.5690 μ s)	▼ 26.40 %
CoSim total exec. @ target clock	1793cyc → 17930.0 ns (17.9300 μ s)	1281cyc → 12810.0 ns (12.8100 μ s)	▼ 28.56 %
CoSim total exec. @ estimated clock	1793cyc → 8925.6 ns (8.9256 μ s)	1281cyc → 6569.0 ns (6.5690 μ s)	▼ 26.40 %

Resource Utilisation

Metric	04_auto-unroll	09_manual-unroll	Δ
LUT	392 (0.74 %)	3676 (6.91 %)	▲ 837.76 %
FF	406 (0.38 %)	1962 (1.84 %)	▲ 383.25 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	2 (0.91 %)	4 (1.82 %)	▲ 100.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			▲ 421.43 %

Power and Energy

Metric	04_auto-unroll	09_manual-unroll	Δ
Static power P_{static}	0.103 W	0.103 W	= 0.00 %
Dynamic power P_{dynamic}	0.002 W	0.013 W	▲ 550.00 %
Total power P_{total}	0.104 W	0.115 W	▲ 10.58 %
Energy/op E_{op}	9.283×10^{-7} J	7.554×10^{-7} J	▼ 18.62 %
Total energy E_{total}	9.283×10^{-7} J	7.554×10^{-7} J	▼ 18.62 %

Figures of Merit

Metric		04_auto-unroll	09_manual-unroll	Δ
Area-Delay Product (ADP)		180.9656	694.4647	▲ 283.76 %
Energy-Delay Product (EDP)		8.2852×10^{-3}	4.9624×10^{-3}	▼ 40.11 %

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	04	09	Δ	04	09	Δ	04	09	Δ
L1	128	128	= 0.00 %	139	42	▼ 69.78 %	—	—	—
L2	64	32	▼ 50.00 %	7	6	▼ 14.29 %	2	1	▼ 50.00 %

FSM Structure

Metric	04_auto-unroll	09_manual-unroll	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	2.770 ns	5.120 ns	▲ 84.84 %

Analisi delle metriche

L'evoluzione è drastica. La soluzione 04 era limitata a un $\mathbb{I} = 2$ a causa di colli di bottiglia simultanei su memorie e sull'accumulatore, impiegando 1793 cicli totali. L'approccio manuale sblocca la vera concorrenza spaziale: l'uso coordinato degli array temporanei e del partizionamento ottimizzato spinge il throughput al limite ideale di $\mathbb{I} = 1$ pur gestendo una

pipeline larga 4 moltiplicatori. Il tempo di calcolo scende a soli 1281 cicli totali, ottenendo il miglior risultato temporale al netto di un evidente ma giustificato innalzamento nell'uso delle LUT (da 392 a 3676) e dei Flip-Flop.

Analisi preliminare

Il secondo confronto isola il solo vantaggio sintattico, confrontando la Soluzione 09 con la migliore variante automatica (Soluzione 07), la quale disponeva già di un corretto e simmetrico partizionamento di memoria, ma si affidava ciecamente allo *scheduler* per le logiche del loop.

Timing

Metric	07_part-v3	09_manual-unroll	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	4.978 ns	5.128 ns	▲ 3.01 %
Max achievable $F_{\max}$	200.880 MHz	195.010 MHz	▼ 2.92 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	07_part-v3	09_manual-unroll	Δ
Latency max @ target clock	17665cyc → 16650.0 ns (16.6500 μ s)	5377cyc → 12810.0 ns (12.8100 μ s)	▼ 23.06 %
Latency max @ estimated clock	17665cyc → 8288.4 ns (8.2884 μ s)	5377cyc → 6569.0 ns (6.5690 μ s)	▼ 20.74 %
CoSim total exec. @ target clock	1665cyc → 16650.0 ns (16.6500 μ s)	1281cyc → 12810.0 ns (12.8100 μ s)	▼ 23.06 %
CoSim total exec. @ estimated clock	1665cyc → 8288.4 ns (8.2884 μ s)	1281cyc → 6569.0 ns (6.5690 μ s)	▼ 20.74 %

Resource Utilisation

Metric	07_part-v3	09_manual-unroll	Δ
LUT	1747 (3.28 %)	3676 (6.91 %)	▲ 110.42 %
FF	1654 (1.55 %)	1962 (1.84 %)	▲ 18.62 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	2 (0.91 %)	4 (1.82 %)	▲ 100.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			▲ 83.94 %

Power and Energy

Metric	07_part-v3	09_manual-unroll	Δ
Static power P_{static}	— W	0.103 W	—
Dynamic power P_{dynamic}	— W	0.013 W	—
Total power P_{total}	— W	0.115 W	—
Energy/op E_{op}	— J	7.554×10^{-7} J	—
Total energy E_{total}	— J	7.554×10^{-7} J	—

Figures of Merit

Metric		07_part-v3	09_manual-unroll	Δ
Area-Delay Product (ADP)		476.3658	694.4647	▲ 45.78 %
Energy-Delay Product (EDP)		—	4.9624×10^{-3}	—

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	07	09	Δ	07	09	Δ	07	09	Δ
L1	128	128	= 0.00 %	138	42	▼ 69.57 %	—	—	—
L2	64	32	▼ 50.00 %	6	6	= 0.00 %	2	1	▼ 50.00 %

FSM Structure

Metric	07_part-v3	09_manual-unroll	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	2.770 ns	5.120 ns	▲ 84.84 %

Analisi delle metriche

Mettendo in paragone le architetture, emerge il puro scoglio semantico della destrutturazione in C++. Mentre la Soluzione 07 rimaneva bloccata a $\text{II} = 2$ (1665 cicli) esclusivamente per la salvaguardia intrinseca della ricorrenza sull'accumulatore `sum`, la separazione esplicita

nell'array temporaneo `partial_sums` permette ad HLS di bypassare i limiti del sintetizzatore. Le somme intermedie vengono disaccoppiate e risolte simultaneamente, e l'architettura scala a 1281 cicli massimizzando pienamente i DSP estratti.

Analisi preliminare

L'ultimo accostamento ha una finalità prettamente diagnostica per validare la necessità dei partizionamenti sulle porte. Contrapponiamo l'esatta sintassi a «Srotolamento Manuale» (fattore 4) processata dapprima senza alcuna direttiva di partizionamento BRAM e successivamente con il corretto partizionamento ciclico a fattore 2.

Timing

Metric	09_no-partition	09_partitioned	Δ
Target clock	10.000 ns	10.000 ns	—
Estimated clock / $F_{\max}$	5.128 ns	5.128 ns	= 0.00 %
Max achievable $F_{\max}$	195.010 MHz	195.010 MHz	= 0.00 %
Clock uncertainty	2.7 ns	2.7 ns	—

Latency and Throughput

Metric	09_no-partition	09_partitioned	Δ
Latency max @ target clock	9473cyc → 14090.0 ns (14.0900 μ s)	5377cyc → 12810.0 ns (12.8100 μ s)	▼ 9.08 %
Latency max @ estimated clock	9473cyc → 7225.4 ns (7.2254 μ s)	5377cyc → 6569.0 ns (6.5690 μ s)	▼ 9.08 %
CoSim total exec. @ target clock	1409cyc → 14090.0 ns (14.0900 μ s)	1281cyc → 12810.0 ns (12.8100 μ s)	▼ 9.08 %
CoSim total exec. @ estimated clock	1409cyc → 7225.4 ns (7.2254 μ s)	1281cyc → 6569.0 ns (6.5690 μ s)	▼ 9.08 %

Resource Utilisation

Metric	09_no-partition	09_partitioned	Δ
LUT	843 (1.58 %)	3676 (6.91 %)	▲ 336.06 %
FF	631 (0.59 %)	1962 (1.84 %)	▲ 210.94 %
BRAM	0 (0.00 %)	0 (0.00 %)	—
DSP	4 (1.82 %)	4 (1.82 %)	= 0.00 %
$A_{\text{norm}} = \Sigma (\text{used} / \text{available})$			▲ 164.58 %

Power and Energy

Metric	09_no-partition	09_partitioned	Δ
Static power P_{static}	— W	0.103 W	—
Dynamic power P_{dynamic}	— W	0.013 W	—
Total power P_{total}	— W	0.115 W	—
Energy/op E_{op}	— J	7.554×10^{-7} J	—
Total energy E_{total}	— J	7.554×10^{-7} J	—

Figures of Merit

Metric	09_no-partition	09_partitioned	Δ
Area-Delay Product (ADP)	288.7106	694.4647	▲ 140.54 %
Energy-Delay Product (EDP)	—	4.9624×10^{-3}	—

Loop Comparison

Loop	Trip count			Iter. latency (cyc)			II		
	09	09	Δ	09	09	Δ	09	09	Δ
L1	128	128	= 0.00 %	74	42	▼ 43.24 %	—	—	—
L2	32	32	= 0.00 %	7	6	▼ 14.29 %	2	1	▼ 50.00 %

FSM Structure

Metric	09_no-partition	09_partitioned	Δ
Number of FSM states	5	5	= 0.00 %
Critical path (max, any state)	5.120 ns	5.120 ns	= 0.00 %

Analisi delle metriche

I dati mostrano gli effetti della limitazione del numero di porte. Se si omette la direttiva `ARRAY_PARTITION`, 4 moltiplicatori vengono istanziati ma forzati a pescare i dati da una singola BRAM blindata al tetto fisico di 2 letture per colpo di clock. Essendo privo dei

percorsi logici necessari, lo scheduler interviene brutalmente per serializzare l'estrazione: l'*Initiation Interval* regredisce inesorabilmente a 2. Di conseguenza, il tempo in co-simulazione subisce un contraccolpo lievitando a 1409 cicli e l'area logica di contorno si riduce (le LUT collassano a 843), fotografando un datapath srotolato ma affetto da stalli prolungati.

Conclusioni

Lo «Srotolamento Manuale» si consacra come uno strumento tecnico obbligatorio nella progettazione HLS in presenza di cicli a limite dinamico incrociati con parallelismo spaziale. Il conseguimento ineccepibile del target operativo ideale ($\mathbb{I} = 1$ a fattore parallelo 4) conferma che la pura ottimizzazione automatica del codice di base non basta. Fatto ciò però la manipolazione architetturale produce esiti concreti solo se spalleggiata da un'infrastruttura di interconnessione speculare, come dimostrato dall'ingegnoso sfruttamento congiunto del partizionamento ciclico e dei canali di lettura nativi **True Dual-Port**. L'unione di queste due visioni (logica e strutturale) spezza l'albero delle dipendenze originarie, garantendo il throughput massimale all'algoritmo *SpMV*.

5. Sintesi dei Risultati e Conclusioni

5.1. Analisi conclusive

Il percorso di esplorazione dello spazio di progettazione (*Design Space Exploration*) condotto sull'algoritmo *SpMV* ha evidenziato in modo chiaro le sfide intrinseche legate all'implementazione hardware di algoritmi caratterizzati da accessi in memoria irregolari e flussi di controllo dipendenti dai dati (come i limiti variabili del ciclo indotti dal formato CRS).

L'analisi progressiva delle metriche ha portato all'emersione di tre concetti architetturali fondamentali:

- **Il muro della memoria (Memory Wall):** Tentare di forzare il parallelismo spaziale o temporale tramite le direttive `#pragma HLS unroll` o `pipeline` senza un'adeguata infrastruttura di memoria porta a un sistematico degrado delle prestazioni. Lo *scheduler* di Vivado HLS, di fronte a un numero insufficiente di porte BRAM, serializza gli accessi, innalzando l'*Initiation Interval* ($\mathbb{I}$) e sprecando le risorse logiche duplicate. Il partizionamento degli array (`cyclic` per gli accessi contigui, `complete` per gli accessi indiretti casuali) si è rivelato un prerequisito non negoziabile.
- **I limiti semantici dell'auto-unrolling:** Anche in presenza di una banda passante ideale, lo srotolamento automatico fallisce nel garantire il massimo *throughput* a causa delle dipendenze algoritmiche dei dati. La ricorrenza *Read-After-Write* (Data Hazard) generata dall'accumulatore scalare `sum` costringe l'hardware a rispettare la sequenzialità semantica imposta dal linguaggio C++, bloccando l'efficienza della pipeline.
- **L'efficacia del Manual Unrolling e del True Dual-Port:** La destrutturazione manuale del nucleo di calcolo si è dimostrata l'unica via per ingannare le limitazioni semantiche. L'introduzione di un array di appoggio temporaneo (`partial_sums`) ha frammentato le dipendenze, permettendo all'hardware di calcolare somme indipendenti. Inoltre, l'allineamento strategico tra il codice srotolato e la natura *True Dual-Port* delle BRAM interne all'FPGA ha permesso di raddoppiare il parallelismo logico mantenendo un fattore di partizionamento fisico minimo, abbattendo drasticamente i costi di *routing* (LUT e FF).

5.2. Conclusioni

L'implementazione dell'algoritmo SpMV tramite Vivado HLS dimostra empiricamente che la Sintesi ad Alto Livello offre strumenti estremamente potenti per l'astrazione hardware, ma non riesce ad essere perfetto in autonomia per il codice software procedurale.

Per ottenere architetture efficienti e ad alte prestazioni, il progettista non può limitarsi all'iniezione cieca di direttive (**pragma**), ma deve adottare un paradigma di programmazione **Hardware-Aware**. È imperativo possedere una profonda comprensione della microarchitettura fisica target (porte di memoria, blocchi DSP, reti di interconnessione) e piegare la sintassi del linguaggio C/C++ per esporre esplicitamente il parallelismo logico desiderato.

La soluzione finale implementata (Soluzione 09), basata sullo srotolamento manuale e sul partizionamento ibrido speculare, rappresenta il culmine di questa metodologia. La macchina generata ha raggiunto il target ideale di $\mathbb{I} = 1$ con un parallelismo operativo di grado 4, domando la natura spigolosa dei *variable bounds* e trasformando un algoritmo nato sequenziale in un datapath massimamente parallelizzato, con un *Trade-off* ottimale tra risorse logiche impiegate e latenza computazionale complessiva.